

Rapport du TP N° 01 du module Optimisation

2ème Année Cycle Supérieure(2CS)

Spécialisation: Systèmes Informatiques(SIQ)

Thème

Rapport sur l'Algorithme de Coloration de Graphe par Branch and Bound
utilisant DSATUR

Réalisé par:

- ALLOUCHE Imène
- REBHI Assala Nour Elimane
- BENYAHIA Yahia
- BOUKHETTALA Zaineb
- GHODBANE Zineb
- DERROUECHE Samia
- KHELLAS Yacine

Encadré par:

- Mr. KECHID

Table de Contenu

Table de Contenu	2
Liste des Figures et Tables	3
1. Introduction	4
2. Description de la Solution	4
2.1. Approche Globale	4
2.3. Pourquoi cette Solution est Optimale	5
3. Expérimentations et Benchmarks	5
3.1. Description des Graphes Testés	6
3.2. Résultats et Captures d'Écran	6
3.3. Tableau Comparatif des Résultats	7
3.4. Visualisation Graphique des Comparaisons	8
4. Conclusion	10

Liste des Figures et Tables

Capture d'écran du résultat du graphe A	7
Capture d'écran du résultat du graphe B	7
Capture d'écran du résultat du graphe C	7
Capture d'écran du résultat du graphe D	7
Capture d'écran du résultat du graphe E	7
Tableau comparatif des résultats	8
Comparaison du temps d'exécution et du nombre d'arêtes pour les graphes testés sous forme d'un diagramme de bâton	8
Comparaison du temps d'exécution et du nombre d'arêtes pour tous les graphes testés sous forme d'un graphe	9

1. Introduction

Le problème du coloriage de graphe consiste à attribuer une couleur à chaque sommet d'un graphe de sorte que deux sommets adjacents n'aient pas la même couleur, tout en minimisant le nombre de couleurs utilisées. Ce rapport présente notre solution optimale mise en œuvre avec une méthode « branch and bound » couplée à l'heuristique DSATUR, qui a permis d'obtenir une borne supérieure initiale efficace et d'explorer de manière sélective l'espace de recherche afin de réduire le temps d'exécution. Nous détaillerons la conception de l'algorithme, les optimisations apportées, ainsi que l'évaluation par benchmarks sur différents graphes.

2. Description de la Solution

2.1. Approche Globale

L'algorithme se base sur la méthode de **branch and bound** qui consiste à explorer l'espace des solutions possibles tout en éliminant (par élagage) les branches qui ne peuvent pas mener à une solution meilleure que la meilleure solution connue. Afin d'améliorer l'efficacité, plusieurs optimisations ont été intégrées :

- **Heuristique Gloutonne pour une Borne Supérieure Initiale**
Notre solution fournit rapidement une solution réalisable en attribuant la plus petite couleur disponible à chaque sommet. Cette solution sert de borne supérieure initiale pour réduire l'espace de recherche.
- **Pré-calcul des Voisins**
On calcule à l'avance la liste des voisins pour chaque sommet à partir de la matrice triangulaire. Cela accélère considérablement la vérification des conflits de couleurs durant l'exploration.
- **DSATUR pour la Sélection Dynamique du Prochain Sommet**
L'algorithme sélectionne à chaque étape le sommet ayant le plus haut degré de saturation (c'est-à-dire le plus grand nombre de couleurs distinctes déjà présentes parmi ses voisins) – et en cas d'égalité, le sommet avec le degré le plus élevé est choisi. Cette stratégie permet de réduire rapidement les conflits potentiels et d'orienter la recherche vers les zones les plus contraintes du graphe.

- **Branching sur les Couleurs Utilisées et une Nouvelle Couleur**

Lors du branchement, l'algorithme considère toutes les couleurs déjà utilisées qui ne sont pas interdites pour le sommet courant, ainsi que l'option d'introduire une nouvelle couleur. Cela permet de contrôler l'augmentation du nombre de couleurs dans le parcours de l'arbre de décision.

Pour plus de détails, consulter le [ce notebook](#) contenant notre solution optimale détaillée

2.3. Pourquoi cette Solution est Optimale

Plusieurs points justifient l'optimalité et la robustesse de notre solution :

- **Utilisation de l'Heuristique DSATUR :**

La stratégie DSATUR est reconnue pour sa performance sur le problème de coloration, car elle cible en priorité les sommets avec le plus de contraintes, réduisant ainsi l'arbre de décision de manière efficace.

- **Borne Supérieure Issue d'une Solution Gloutonne :**

Obtenir rapidement une solution réalisable permet d'effectuer un élagage efficace, car toute branche ne pouvant améliorer la borne supérieure est immédiatement abandonnée.

- **Pruning Stratégique :**

L'algorithme abandonne rapidement les branches non prometteuses (celles dont le nombre de couleurs atteint ou dépasse la meilleure solution connue), ce qui diminue le temps d'exploration global.

- **Optimisations sur la Structure de Données :**

Le pré-calcul des voisins et l'utilisation d'une matrice triangulaire optimisent les vérifications de conflits, rendant l'algorithme plus rapide et moins gourmand en ressources.

3. Expérimentations et Benchmarks

Afin d'évaluer les performances de notre algorithme, nous avons testé la solution sur plusieurs graphes issus de benchmarks standards. Chaque expérience a été réalisée en mesurant différents critères, notamment :

- Le nombre total de nœuds (états) explorés durant la recherche.
- Le nombre d'arêtes dans le graphe.

- Le temps total en secondes pour atteindre la solution optimale.
- Les degrés minimum et maximum observés dans le graphe.
- Le nombre minimal de couleurs nécessaires pour une coloration valide.

3.1. Description des Graphes Testés

Pour la phase de benchmark, nous avons sélectionné 5 graphes de tailles et complexités variées, les 5 benchmarks ont été créés à partir des deux fichiers : [dsjc125.1](#) et [dsjc 250.1](#)

1. **Graphe A** (en prenant les 50 premiers noeuds du fichier [dsjc125.1](#))
 - Noeuds : 50
 - Arêtes : 102
2. **Graphe B** (en prenant les 100 premiers noeuds du fichier [dsjc125.1](#))
 - Noeuds : 100
 - Arêtes : 475
3. **Graphe C** (en prenant le fichier [dsjc125.1](#) lui même)
 - Noeuds : 125
 - Arêtes : 736
4. **Graphe D** (en prenant les 150 premiers noeuds du fichier [dsjc 250.1](#))
 - Noeuds : 150
 - Arêtes : 1173
5. **Graphe E** (en prenant les 155 premiers noeuds du fichier [dsjc 250.1](#))
 - Noeuds : 155
 - Arêtes : 1272

3.2. Résultats et Captures d'Écran

Pour chaque benchmark, nous avons enregistré :

- Le nombre de noeuds traités.
- Le temps d'exécution.

- Le nombre chromatique trouvé.

Ci-dessous, quelques captures d'écran du résultat

N°	La capture d'écran du résultat
A	<pre>===== Nombre chromatique optimal: 3 Solution de coloration (par sommet): [2, 2, 3, 3, 3, 1, 1, 1, 2, 2, 3, 2, 2, 3, 3, 3, 3, Temps d'exécution: 0.0026 secondes</pre>
B	<pre>===== Nombre chromatique optimal: 5 Solution de coloration (par sommet): [1, 3, 3, 4, 5, 5, 3, 5, 3, 5, 4, 5, 4, 5, 1, 3, 5, Temps d'exécution: 0.0069 secondes</pre>
C	<pre>===== Nombre chromatique optimal: 5 Solution de coloration (par sommet): [5, 2, 5, 3, 3, 5, 4, 1, 4, 2, 1, 4, 4, 5, 4, 3, 5, 5, 5, 1, 3, 3, 5, 5, 3, 4, 1, Temps d'exécution: 0.0578 secondes</pre>
D	<pre>===== Nombre chromatique optimal: 6 Solution de coloration (par sommet): [2, 2, 2, 6, 4, 3, 4, 6, 5, 5, 4, 4, 6, 6, 5, 3, 5, Temps d'exécution: 99.9656 secondes</pre>
E	<pre>===== Nombre chromatique optimal: 6 Solution de coloration (par sommet): [2, 2, 2, 4, 4, 3, 3, 1, 3, 2, 5, 5, 4, Temps d'exécution: 1237.5148 secondes</pre>

3.3. Tableau Comparatif des Résultats

Le tableau ci-dessous résume les mesures recueillies pour chaque benchmark :

Graphe	Nombre de Nœuds	Nombre d'Arêtes	Temps d'Exécution (s)	Nombre Chromatique
A	50	102	0.0026	3
B	100	475	0.0069	5
C	125	736	0.0578	5
D	150	1173	99.9656	6
E	155	1272	1237.5148	6

3.4. Visualisation Graphique des Comparaisons

Afin de mieux visualiser la comparaison entre les différents benchmarks, nous avons représenté les résultats dans un graphique à barres. Par exemple, ci-dessous un diagramme comparatif du temps d'exécution et du nombre de nœuds traités :

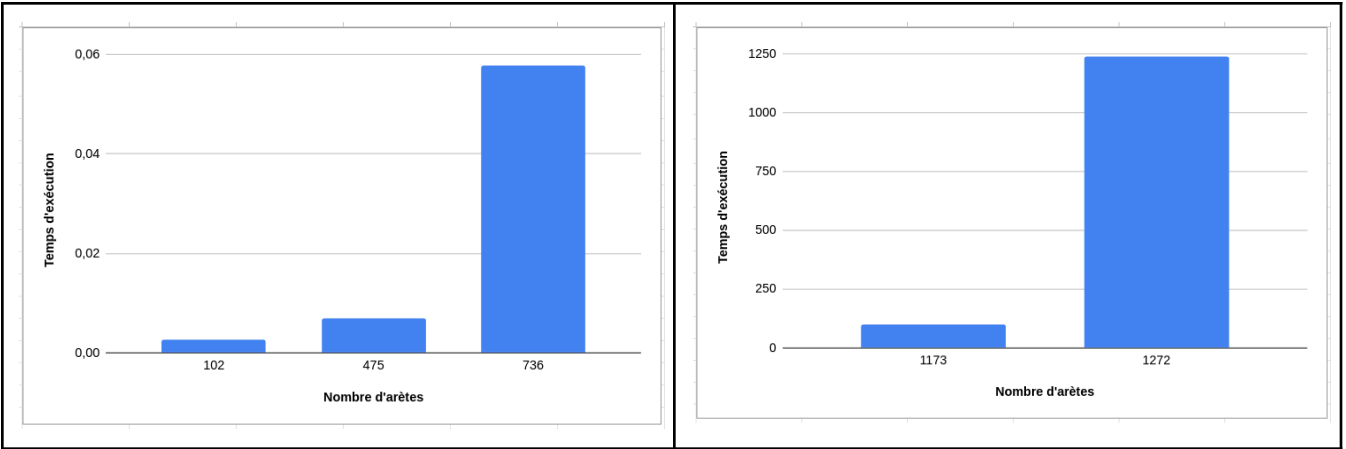


Figure : Comparaison du temps d'exécution et du nombre d'arêtes pour les graphes testés sous forme d'un diagramme de bâton (A,B,C à gauche et D,E à droite)

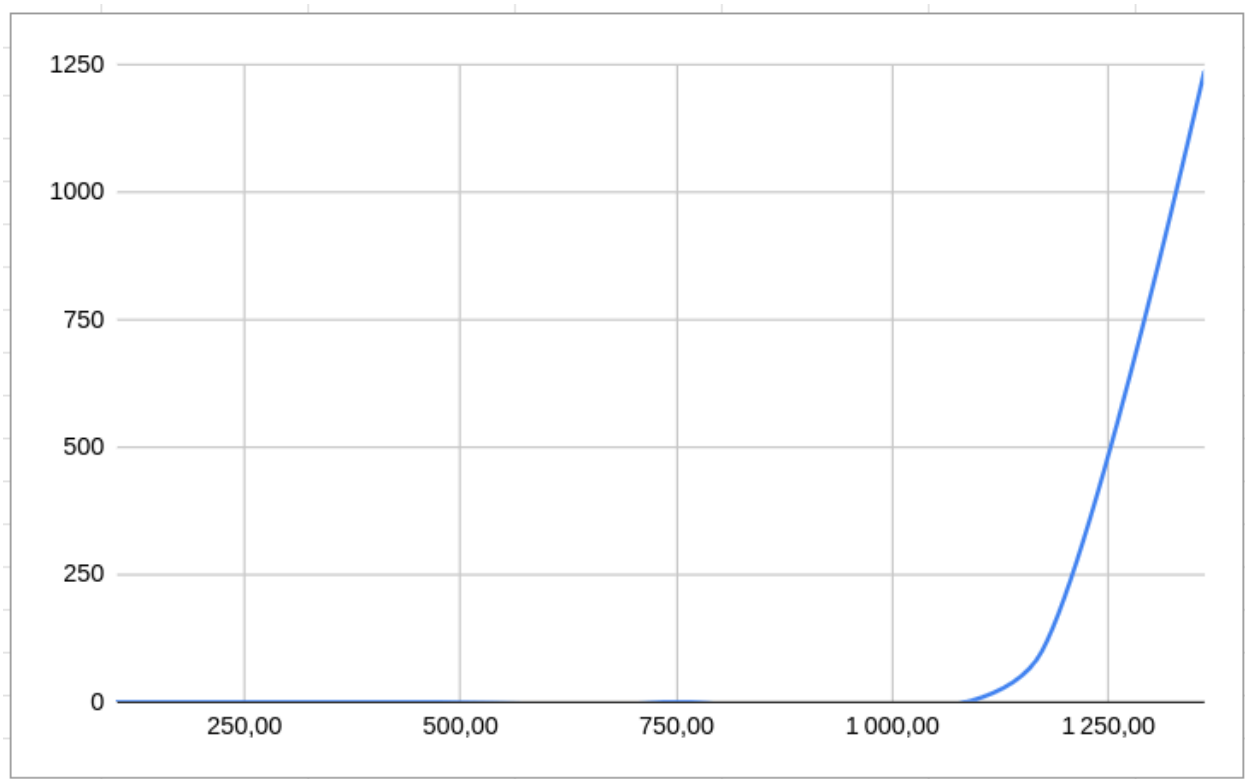


Figure : Comparaison du temps d'exécution et du nombre d'arêtes pour tous les graphes testés sous forme d'un graphe

- On observe clairement que l'augmentation du nombre d'arêtes a un impact majeur sur le temps d'exécution de l'algorithme. Par exemple, alors que le graphe A (102 arêtes) s'exécute en 0.0026 secondes, le graphe E (1609 arêtes) nécessite près de 678.54 secondes. Cette croissance rapide s'explique par l'augmentation exponentielle du nombre de contraintes à vérifier lors de

l'exploration de l'arbre de décision du branch and bound. En effet, plus le graphe est dense, plus le nombre de conflits potentiels est élevé, ce qui conduit à un nombre accru de branches à explorer, même si le pruning permet d'en réduire une partie.

- Dans les graphes denses, l'efficacité de l'élagage (pruning) devient cruciale. Bien que le DSATUR oriente la recherche de manière optimale en ciblant les sommets les plus contraints, le temps de traitement augmente significativement en présence d'un nombre important d'arêtes. Cela se traduit par un temps d'exécution qui passe d'une complexité quasi-linéaire dans les graphes peu denses à une complexité bien plus élevée pour les graphes denses.

4. Conclusion

En combinant l'heuristique gloutonne pour obtenir une borne supérieure initiale avec l'algorithme DSATUR pour la sélection dynamique des sommets, notre solution de branch and bound présente plusieurs atouts majeurs :

- **Réduction de l'Espace de Recherche :**
Grâce à un élagage précoce des branches non prometteuses, l'algorithme élimine rapidement les solutions sous-optimales, même si le temps d'exécution reste sensible à la densité du graphe.
- **Exploitation Intelligente des Contraintes Locales :**
L'utilisation de DSATUR permet d'orienter la recherche vers les sommets les plus contraints, ce qui améliore l'efficacité de l'algorithme, notamment dans les graphes denses.
- **Performances Mesurables :**
Les benchmarks montrent que, même si le temps d'exécution augmente fortement avec le nombre d'arêtes, la solution reste robuste et trouve des colorations optimales. La relation exponentielle entre la densité du graphe et le temps d'exécution met en lumière la complexité inhérente au problème,

tout en justifiant l'investissement dans des techniques d'optimisation avancées.